

TEMA 90. ANÁLISIS DINÁMICO DE SISTEMAS: MODELADO DE PROCESOS, MODELADO DINÁMICO Y BPMN (BUSINESS PROCESS MODEL AND NOTATION).

Actualizado a 27/04/2019

Contenido

1	Cambios en contenidos y esquemas.....	3
2	Análisis estructurado	3
2.1	Análisis esencial.....	4
2.2	Técnicas aplicables al análisis estructurado.....	4
2.2.1	Diccionario de datos	4
2.2.2	Diagrama de Flujo de Datos.....	5
2.2.3	Historia de la vida de las entidades (HVE)	8
3	UML.....	9
3.1	Estándares UML 2.0.....	9
3.2	UML en MétricaV3	9
3.2.1	CASOS DE USO.....	9
3.2.2	DIAGRAMA DE CLASES.....	10
3.2.3	Diagrama de objetos.....	12
3.2.4	Diagrama de estructura compuesta (UML 2.X)	13
3.2.5	DIAGRAMA DE COMPONENTES.....	13
3.2.6	DIAGRAMA DE DESPLIEGUE (o diagrama de distribución)	14
3.2.7	DIAGRAMA DE INTERACCIÓN	15
3.2.8	Diagramas de Revisión de interacciones (UML 2.4.X)	17
3.2.9	DIAGRAMA DE TIEMPOS (UML 2.X).....	17
3.2.10	Diagrama de Global de Interacción (UML 2.X)	18
3.2.11	DIAGRAMA DE PAQUETES	18
3.2.12	DIAGRAMA DE TRANSICIÓN DE ESTADOS (UML 2.X Diagrama de Máquina de Estados)	19
3.2.13	Diagrama de Actividad.....	21
3.3	UML 2.X	23
3.4	Mecanismos de extensibilidad	25
3.5	Elementos comunes a cualquier diagrama UML.....	26
4	BPM.....	27
4.1	BPMN.....	28
4.2	XPDL (XML Process Definition Language).....	28
4.3	Cadena de valor BPMN-XPDL-BPEL.....	28
4.4	Lenguajes de orquestación.....	29

1 CAMBIOS EN CONTENIDOS Y ESQUEMAS

Este tema no ha sufrido cambios.

Se elabora el resumen express en formato textual para completar los esquemas disponibles, así como los audios tanto del tema como del resumen express.

2 ANÁLISIS ESTRUCTURADO

Se puede considerar que el análisis estructurado está compuesto por el conjunto de técnicas siguientes:

- ✓ **Diagrama de Flujos de Datos (DFD).** Este diagrama describe cómo fluyen los datos y cómo son estos transformados en el sistema.
- ✓ **Diagrama de Flujos de Control (DFC).** Diagrama similar al DFD pero para modelar los procesos y flujos de control existentes en el sistema. No todos los sistemas requieren la confección de DFCs.
- ✓ **Diccionario de Requisitos o Diccionario de Datos.** Es un listado organizado de todos los elementos de datos que son pertinentes para el sistema, con definiciones precisas y rigurosas que permiten que el usuario y el analista tengan una misma comprensión de las entradas, salidas, de las componentes de los almacenes y de los cálculos intermedios.
- ✓ **Especificaciones de Proceso (EP) o Narrativa de Procesamiento.** Describe los procesos del modelo de flujo de datos que aparecen en el nivel final del refinamiento de los DFDs. Cuando un proceso ya no puede expandirse en un DFD nuevo se procede a confeccionar una especificación de proceso que detalle las actividades que se llevan a cabo en el mismo.
- ✓ **Especificaciones de Control (EC) o Narrativa de Control.** Describen el comportamiento del sistema pero sin entrar en el detalle en el funcionamiento interno de los procesos. Describe cómo se comporta el sistema, activando determinados procesos, cuando se dan ciertas condiciones.
- ✓ **Diagrama de Transición de Estados (DTE).** Representa el comportamiento de un sistema que muestra los estados y los sucesos que hacen que el sistema cambie de estado. Adicionalmente, el DTE especifica las acciones que se llevan a cabo como consecuencia de la ocurrencia de un suceso determinado.
- ✓ **Tabla de Activación de Procesos (TAP).** Representa la información contenida en el Diagrama de Transición de Estados en lo que concierne al contexto de los procesos, no de los estados. La tabla indica qué procesos serán invocados cuando se produzca un suceso.
- ✓ **Diagrama Entidad-Relación.** Permite representar las entidades de datos que existen en el sistema, los atributos de las mismas y las relaciones existentes entre las diferentes entidades.

Partición Funcional, modelizada a través del Diagrama de Flujo de Datos y la Especificaciones de procesamiento.

Partición Estática, modelizada a través del Modelo Entidad – Relación.

Partición Dinámica, modelizada a través de los Diagramas de Transición de Estados y las Tablas de Activación de Procesos

2.1 ANÁLISIS ESENCIAL

El Análisis Esencial constituye una de las primeras actividades del desarrollo de un proyecto y en ella se confecciona un conjunto de modelos del sistema que describen a éste desde la esencia del mismo, es decir, describen lo que el sistema hace o debe hacer, pero de forma independiente a cómo se va a hacer físicamente.

Durante el análisis esencial se construyen los modelos esenciales del sistema, los cuales también se conocen por modelos lógicos o modelos conceptuales.

En contraposición con el modelo esencial está el modelo de implantación, que también se conoce como modelo tecnológico o modelo físico. Este modelo muestra lo que el sistema hace y cómo lo hace, es decir, cómo se implanta físicamente.

El modelo esencial está compuesto de dos partes bien diferenciadas:

- ✓ El Modelo Ambiental, que define la frontera entre el sistema y el exterior.
 - Descripción Breve del Propósito del Sistema
 - Diagrama de Contexto
 - Lista de Eventos (orientados a flujo, temporales, de control)
 - Descripción del Diccionario de Datos Inicial
- ✓ El Modelo de Comportamiento, que define cómo se comporta el sistema cuando interactúa con el ambiente, es decir, con el exterior.
 - Se debe describir qué hace el sistema, pero sin entrar en el detalle de cómo hacerlo.
 - Para construir el modelo de comportamiento se parte del modelo ambiental, del diagrama de contexto, y se va expandiendo en sucesivos diagramas de nivel inferior.
 - Un Conjunto de Diagramas de Flujo de Datos (subsistema y función primitiva)
 - Diccionario de Datos
 - Especificaciones de Proceso

2.2 TÉCNICAS APLICABLES AL ANÁLISIS ESTRUCTURADO

2.2.1 DICcionario DE DATOS

Contendrá todas las definiciones de los flujos de datos existentes en los diferentes DFDs confeccionados. es una técnica que permite especificar el significado y la composición de los datos.

Hay que distinguir dos conceptos diferentes:

- ✓ Diccionario de Datos (DD): Es el conjunto de todas las entradas del diccionario de datos.
- ✓ Entrada del Diccionario de Datos (DDE): Es cada una de las entradas que contiene el diccionario con una definición de un dato.

Características:

- ✓ Los datos del diccionario se definen una sola vez, aún cuando se usen en varios sitios
- ✓ Toda definición de un dato debe describir cómo se compone ese dato y el conjunto de valores que éste puede tomar.
- ✓ Un diccionario es útil cuando es la única fuente de información que se usa.
- ✓ Una de las notaciones que más se usa en la notación BNF (Backus-Naur Form)

2.2.1.1 TIPOS DE DICcionario

- ✓ Diccionarios Activos. Fuerzan a los programadores a utilizar las definiciones del diccionario. Estos diccionarios están conectados con el compilador para buscar la definición de los datos y si se modifica un dato forzar a recompilar.

- ✓ **Diccionarios Pasivos.** Permiten que los programadores puedan definir datos dentro de sus programas y que éstos no estén en el diccionario de datos.
- ✓ **Diccionarios de Recursos de Información** → son una ampliación del concepto de Diccionario de Datos. Los SDRI no sólo incorporan los datos correspondientes al sistema que se está construyendo sino que incorporan todos los detalles de toda la información que fluye en la organización

2.2.2 DIAGRAMA DE FLUJO DE DATOS

Mv3: El objetivo del diagrama de flujo de datos es la **obtención de un modelo lógico de procesos** que represente el sistema, con independencia de las restricciones físicas del entorno.

La técnica de los DFDs permite realizar un análisis descendente del sistema, particionando éste, de forma gráfica e iterativa hasta alcanzar el nivel más bajo del sistema, en donde el proceso a realizar se puede describir con una frase.

En el nivel más alto hay un DFD, al que se denomina DFD de nivel 0 o diagrama de contexto, que representa el sistema completo y su relación con el entorno

2.2.2.1 COMPONENTES DE UN DFD

Los componentes principales de un DFD son los siguientes:

- ✓ **Procesos:** Representan las transformaciones que se realizan sobre los datos en el sistema.
- ✓ **Almacenes de Datos:** Representan la información que almacena el sistema. Son depósitos de datos en los que se almacenan éstos para su uso por uno o varios procesos. Pueden ser tan simples como una memoria intermedia o una cola, o tan sofisticados como una base de datos relacional.
- ✓ **Flujos de Datos:** Representan cómo fluyen los datos dentro del sistema.
- ✓ **Entidades Externas:** Representa los elementos externos al sistema que generan o consumen información generada o producida por el sistema

2.2.2.2 REGLAS

- ✓ **Entidades externas:**
 - Puede aparecer varias veces en un mismo diagrama, así como en los distintos niveles del DFD para mejorar la claridad del diagrama.
 - Las entidades externas se representan sólo en el diagrama de contexto, una vez representadas no es necesario volverlo a hacer en diagramas de nivel inferior
 - Son Generadores y consumidores de Información
- ✓ **Proceso**
 - El proceso nunca es el origen ni el final de los datos, puede transformar un flujo de datos de entrada en varios de salida y siempre es necesario como intermediario entre una entidad externa y un almacén de datos
 - El proceso no puede recibir datos que no se usen para transformación
- ✓ **Almacén**
 - El almacén no puede crear, transformar o destruir datos, no puede estar comunicado con otro almacén o entidad externa y aparecerá por primera vez en aquel nivel en que dos o más procesos accedan a él.
 - Para mejorar la legibilidad se puede representar un almacén en un mismo diagrama varias veces.
 - Como norma general, un almacén debe ser representado en el nivel más bajo posible de la jerarquía de DFDs.
- ✓ **Flujo de datos**

- Un flujo de datos entre dos procesos sólo es posible cuando la información es síncrona, es decir, el proceso destino comienza cuando el proceso origen finaliza su función.
- **No puede crear ni destruir los datos**
- **No activa procesos**
- De consulta: representan la utilización de los valores de uno o más campos de un almacén o la comprobación de que los valores de los campos seleccionados cumplen unos criterios determinados.
- De actualización: representan la alteración de los datos de un almacén como consecuencia de la creación de un nuevo elemento, por eliminación o modificación de otros ya existentes.
- De diálogo: es un flujo entre un proceso y un almacén que representa una consulta y una actualización.
- **Flujos discretos**. Representan datos en movimiento en un momento determinado en el tiempo.
- **Flujos continuos**. Representan flujos de datos persistentes en el tiempo (se representan con doble flecha)

Origen	Destino		
		Proceso	Almacén
	Proceso	X	X
	Almacén	X	-
	Entidad Externa	X	-

- ✓ **Proceso de control**: representa procesos que coordinan y sincronizan las actividades de otros procesos del diagrama de flujo de datos.
- ✓ **Flujo de control**: representa el flujo entre un proceso de control y otro proceso. El flujo de control que sale de un proceso de control activa al proceso que lo recibe y el que entra le informa de la situación de un proceso. los flujos de control no representan datos con valores, sino que en cierto modo, se trata de eventos que activan los procesos (señales o interrupciones).

2.2.2.3 AMPLIACIONES DE WARD Y MELLOR

- ✓ **Flujos de Datos Continuos**: Permite representar los flujos de datos que entran o salen de un proceso de forma continua
- ✓ **Procesos de Control**: Similares a los procesos pero se encargan de procesar señales de control en vez de flujos de datos. Se encargan de coordinar y sincronizar las actividades de otros procesos DFD.
- ✓ **Flujos de Control**. Llevan la información de las señales de control. El flujo de control sale de un proceso de control y activa al proceso convencional al que llega
- ✓ **Almacén de Control**. Similares a los almacenes, pero almacenan la información de control
- ✓ **Procesos Múltiples**. Permite representar varias instancias de un mismo proceso dentro del sistema

Subdivisión jerárquica del DFD

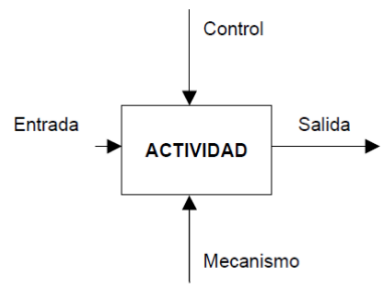
- ✓ Nivel 0: Diagrama de Contexto
- ✓ Nivel 1: Diagrama de Subsistemas
- ✓ Nivel 2: Diagrama de Funciones de los Subsistemas
- ✓ Nivel 3: Diagrama de Subfunciones de las Funciones
- ✓ Nivel 4: Diagrama de Procesos

2.2.2.4 TÉCNICAS PARA MEJORAR Y PROBAR EL DFD

- ✓ Pruebas de corrección. (Exactitud)
- ✓ Pruebas de utilidad (comprensión). → DFD legible.
- ✓ Pruebas de cohesión → obtener una versión mejorada del DFD

2.2.2.5 TÉCNICAS ALTERNATIVAS/COMPLEMENTARIAS AL DFD

- ✓ **SRD:** Structured Requirements Definition
 - Obtención de las entidades externas a partir de los DFD de usuario. Complementario al DFD.
- ✓ **SASS:** Structured Analysis and System Specification
- ✓ **SSD (Structured System Design)** → es una técnica de diseño pero lo meto aquí para relacionarlo con los DFD's
 - El método SSD es un método de diseño que permite obtener la estructura de un sistema software a partir de un DFD que describa su funcionamiento.
 - El objetivo es obtener estructuras de diseño que faciliten el mantenimiento
- ✓ **SADT:** Structured Analysis and Design Technique. → permite representar las actividades de un proceso, definir las dependencias y relaciones entre dichas actividades, los controles que determinan o limitan su ejecución, los mecanismos que los ponen en marcha, así como los datos que se utilizan, comparten o transforman en los procesos.



Existen cinco tipos de interconexiones entre actividades, que son las siguientes:

- Control.
- Entrada.
- Control – Realimentación.

DFD resultado de una explosión, comprobar que:

- ✓ No falten flujos de datos de entrada o salida que acompañaban al proceso del nivel superior.
- ✓ No aparezca algún flujo que no estuviese ya asociado al proceso de nivel superior.
- ✓ Todos los elementos del DFD resultante deben estar conectados directa o indirectamente con los flujos del proceso origen.
- ✓ Comprobación de que todos los flujos de datos que entran y salen de un proceso son los mismos tanto en el nivel donde se define dicho proceso como en el nivel n+1 donde se expande dicho proceso en un nuevo diagrama.
- ✓ Comprobación de que todos los flujos de datos usados en los diagramas se encuentran definidos en el diccionario de datos
- ✓ **Balanceo.** Las comprobaciones son:
 - Comprobar que todos los flujos de entrada al diagrama de nivel n+1 aparecen en el nivel n.
 - Comprobar que todos los flujos de salida del diagrama del nivel n+1 aparecen en el nivel n.
 - Cada flujo de datos y cada almacén de datos deben estar definidos en el diccionario de datos
 - Cada término que aparece en la especificación de proceso debe existir en el diccionario de datos
 - Cada burbuja del DFD debe asociarse con un DFD de nivel inferior o con una especificación de proceso, pero no ambos
- ✓ **Comprobación con Modelo E/R**

- Comprobar que todos los almacenes definidos en el Modelo de Procesos se corresponden con una parte del Modelo de E/R
- Comprobar que todas las entidades del Modelo E/R son accedidas por un proceso primitivo.

2.2.2.6 ESPECIFICACIÓN DE PROCESOS ELEMENTALES

Cuando un proceso ya no puede subdividirse en un nuevo diagrama se dice que se trata de un proceso elemental (proceso primitivo)

Es una técnica que define el procedimiento que realiza un proceso primitivo (lógica de los procesos primitivos)

Debe describir de una manera “formal” cómo se obtienen los flujos de datos de salida a partir de los de entrada más quizás una información local del proceso (cómo se transforman los datos de entrada en los de salida).

Representaciones empleadas:

- ✓ Lenguaje estructurado
- ✓ Narrativa tradicional
- ✓ Árboles de decisión
- ✓ Tablas de decisión
- ✓ Diagramas de acción
- ✓ Pre y post condiciones

2.2.2.7 ELABORAR EL DFD:

1. Elaborar el DFD de Contexto
2. Expandir la Burbuja del DFD de Contexto
3. Expandir la Burbuja de Nivel N para Crear el Diagrama de Nivel N+1
4. Elaborar las Especificaciones de Proceso para los procesos Elementales

2.2.3 HISTORIA DE LA VIDA DE LAS ENTIDADES (HVE)

Describe la evolución de las entidades de datos del sistema en el tiempo.

Dos visiones de los datos.

- ✓ El análisis estático de los datos: interrelaciones entre las entidades de datos del sistema (DED).
- ✓ El análisis dinámico de los datos: muestra el proceso de los flujos de datos. (DFD).
- ✓ La historia de la vida de las Entidades: Verifica la exactitud de los modelos (DED y DFD)
 - Garantiza la coherencia entre las tres visiones del sistema

Elementos

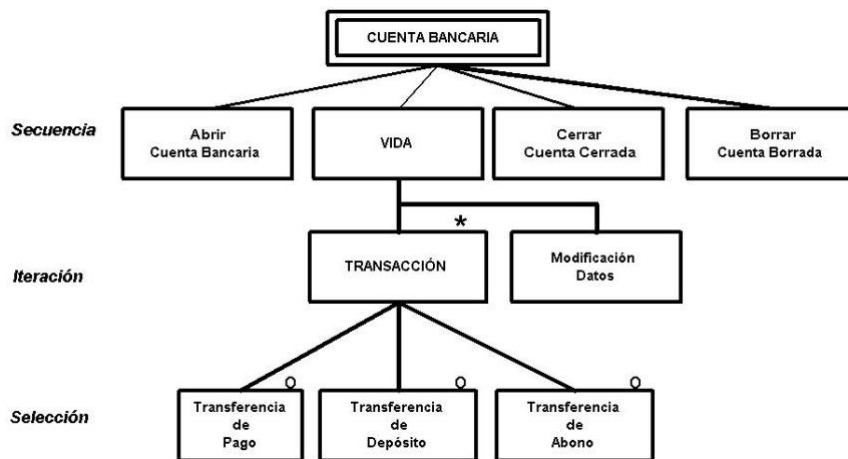
- ✓ Entidades de datos: - Objeto sobre el que el sistema guarda información. → Rectángulo
- ✓ Eventos - Cualquier suceso que activa a un proceso que actualiza datos del sistema
- ✓ Efectos - Describen el resultado de la acción del evento sobre una Entidad determinada
- ✓ Nodo: Agrupa un conjunto de eventos
- ✓ Cajas vacías: Ningún evento de los considerados afecta a la Entidad

La HVE se representa de forma jerárquica. (Parte superior con la entidad cuyo ciclo se va a representar)

Tres tipos de estructuras:

- ✓ Secuencial: Sucesión temporal de eventos, se representa mediante un rectángulo.

- ✓ Selección: Diferentes eventos alternativos, se representa mediante un círculo (o) sobre un rectángulo.
- ✓ Iteración: Repetición de un evento, se representa mediante un asterisco (*) sobre un rectángulo.



3 UML

3.1 ESTÁNDARES UML 2.0

- ✓ Superestructura: Es la especificación que usamos todos los días. Aquí se encuentran todos los diagramas que la mayoría de los desarrolladores conocen.
- ✓ Infraestructura: Conceptos de bajo nivel. Meta-Modelo da soporte a la superestructura, entre otras.
- ✓ OCL: Lenguaje de restricción de Objetos. De utilidad para especificar conceptos ambiguos sobre los distintos elementos del diagrama.
- ✓ XMI/Intercambio de diagramas. Permite compartir diagramas entre herramientas de modelado

3.2 UML EN MÉTRICAV3

3.2.1 CASOS DE USO

Capturar los requisitos funcionales del sistema y expresarlos desde el punto de vista del usuario (proporcionan una visión de "caja negra").

Guiar todo el proceso de desarrollo del sistema de información.

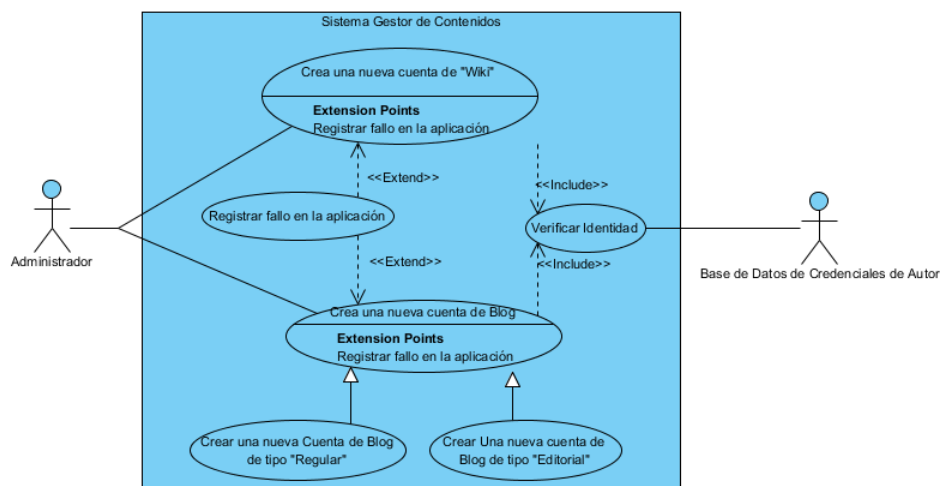
Elementos fundamentales:

- ✓ **Actores**. Un actor es algo o alguien que se encuentra fuera del sistema y que interactúa con él. (Primarios, secundarios e iniciadores)
- ✓ **Casos de uso**. Un caso de uso representa el comportamiento que ofrece el sistema de información desde el punto de vista del usuario

- ✓ **Relaciones:** Las relaciones pueden tener lugar **entre actores y casos de uso** (comunicaciones).
Puedes unidireccional o bidireccional
 - **entre casos de uso.** Es unidireccional
 - La relación “usa” (es de inclusión <<include>>) se utiliza cuando se quiere reflejar un comportamiento común en varios casos de uso
 - La relación “extiende” (es de extensión) se utiliza cuando se quiere reflejar un comportamiento opcional de un caso de uso.
 - Generalización Este tipo de relaciones se pueden dar tanto entre casos de uso como entre actores.
 - un caso de uso hereda el comportamiento y significado de otro.
- ✓ **Contexto del sistema (SystemBoundary):** Cuadro que contiene las diferentes partes del sistema y los casos de uso que la forman

Tipos de casos de uso

- **Primario:** El caso de uso representa una funcionalidad básica del sistema.
- **Secundario:** El caso de uso es un proceso menor o de rara ejecución.
- **Opcional.** El caso de uso puede no ser considerado.
- **Esencial:** Un caso de uso cuya ejecución depende muy poco del estado de la tecnología o de los detalles de implementación. hacen referencia a la funcionalidad esperada
- **Implementación (o real o concreto.** Describe un proceso dependiente de la implementación. hacen referencia a detalles de la interfaz



3.2.2 DIAGRAMA DE CLASES

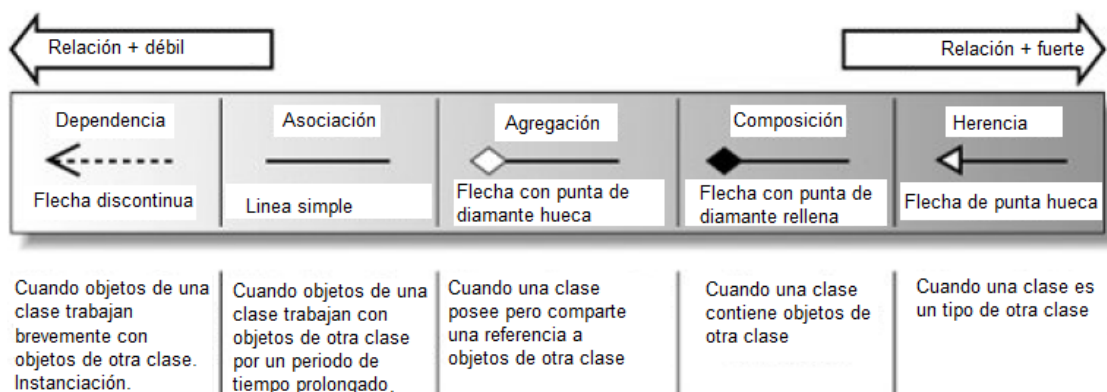
El objetivo principal de este modelo es la representación de los aspectos estáticos del sistema, utilizando diversos mecanismos de abstracción (clasificación, generalización, agregación)

El diagrama de clases recoge las clases de objetos y sus asociaciones. En este diagrama se representa la estructura y el comportamiento de cada uno de los objetos del sistema y sus relaciones con los demás objetos, **pero no muestra información temporal**

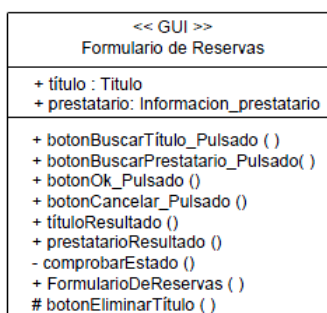
Elementos

- ✓ **Clases** Una clase describe un conjunto de objetos con propiedades (atributos) similares y un comportamiento común
 - Las clases y en general todos los elementos de los diagramas, pueden estar clasificados de acuerdo a varios criterios, como por ejemplo su objetivo dentro de un programa. Esta clasificación adicional se expresa mediante un **Estereotipo**.
 - Algunos de los autores de métodos OO, establecen una clasificación de todos los objetos que pueden aparecer en un modelo. Los tipos son: – Objetos Entidad. – Objetos límite o interfaz. – Objetos de control.
 - Éstos son estereotipos de clases. Un estereotipo representa la meta-clasificación de un elemento.
- ✓ **Relaciones**
 - La multiplicidad es una restricción que se pone a una relación y que limita el número de instancias de una clase que pueden tener esa relación con una instancia de la otra clase
 - Asociación. Las relaciones de asociación representan un conjunto de enlaces entre objetos o instancias de clases. Es el tipo de relación más general, y denota básicamente una dependencia semántica.
 - Elementos adicionales que doten de mayor detalle al tipo de relación:
 - Rol, o nombre de la asociación, que describe la semántica de la relación en el sentido indicado.
 - Multiplicidad, que describe la cardinalidad de la relación, → Uno a uno, uno a muchos y muchos a muchos.
 - Herencia (es-un). Las jerarquías de generalización/especialización se conocen como herencia.
 - permite a una clase de objetos incorporar atributos y métodos de otra clase, añadiéndolos a los que ya posee.
 - La generalización define una superclase a partir de otras.
 - Estereotipo de generalización. Implementation: La subclase hereda la implementación de la superclase, sin publicar ni soportar sus interfaces.
 - Restricciones de generalización.
 - Complete: La generalización ya no permite mas subclases.
 - Incomplete: Se puede incorporar más subclases a la generalización.
 - Disjoint: Sólo se puede tener un tipo en tiempo de ejecución, es decir, una instancia de la superclase solo podrá ser de un tipo de subclase.
 - Overlapping: Se puede cambiar de tipo durante su vida, o sea, una instancia de la superclase puede ir cambiando de tipo entre los de sus subclases.
 - La especialización o especificación es la operación inversa, y en ella una clase se descompone en una o varias subclases.
 - Agregación (son-parte-de). La agregación es un tipo de relación jerárquica entre un objeto que representa la totalidad de ese objeto y las partes que lo componen.
 - Composición. La composición es una forma de agregación donde la relación de propiedad es más fuerte, e incluso coinciden los tiempos de vida del objeto completo y las partes que lo componen.

- **Dependencia.** Una relación de dependencia se utiliza entre dos clases o entre una clase y una interfaz, e indica que una clase requiere de otra para proporcionar alguno de sus servicios. UML permite dar más significado a las dependencias, es decir concretar más, mediante el uso de estereotipos.
 - Bind - se indica que la clase se instancia con los parámetros pasándole datos reales para sus parámetros.
 - Derive: Se utiliza para indicar relaciones entre dos atributos e indica que el valor de uno depende directamente del valor de otro.
 - Friend: Especifica una visibilidad especial sobre la clase relacionada. Es decir, podrá ver las interioridades de la clase destino.
 - InstanceOF: Indica que el objeto origen es una instancia del destino
 - Instantiate: indica que el origen crea instancias del destino.
 - Powertype: indica que el destino es un contenedor de objetos del origen, o de sus hijos.
 - Refine: se utiliza para indicar que una clase es la misma que otra, pero más refinada, es decir dos vistas de la misma clase, la destino con mayor detalle.



- ✓ **Interfaces** Una interfaz es una especificación de la semántica de un conjunto de operaciones de una clase o paquete que son visibles desde otras clases o paquetes. Normalmente, se corresponde con una parte del comportamiento del elemento que la proporciona.
- ✓ **Paquetes** Los paquetes se usan para dividir el modelo de clases del sistema de información, agrupando clases u otros paquetes según los criterios que sean oportunos. Las dependencias entre ellos se definen a partir de las relaciones establecidas entre los distintos elementos que se agrupan en estos paquetes



Esquema de una clase:

En la zona superior se muestra el nombre de la clase y propiedades generales como el estereotipo.

La zona central contiene una lista de atributos, uno en cada línea

Forma parte de la vista estática del sistema. En este diagrama se modelan las instancias de las clases del diagrama de clases. Muestra los objetos y sus relaciones, pero en un momento concreto del sistema. Estos diagramas contienen objetos y enlaces. En los diagramas de objetos también se pueden incorporar clases para mostrar la clase de la que es representado un objeto.

3.2.4 DIAGRAMA DE ESTRUCTURA COMPUESTA (UML 2.X)

Muestra la estructura interna de una clase (visión de la estructura como una caja blanca), incluyendo sus puntos de interacción con otras partes del sistema (comunicación entre estructuras).

Utilizaremos este tipo de diagramas cuando necesitemos mostrar una relación inusual entre elementos, algo que no podamos representar en un diagrama de clases.

Diagrama de clases

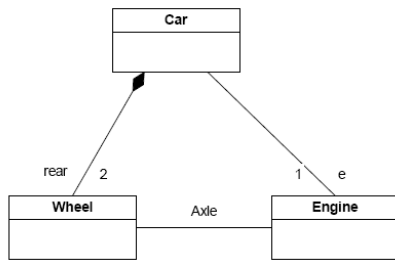
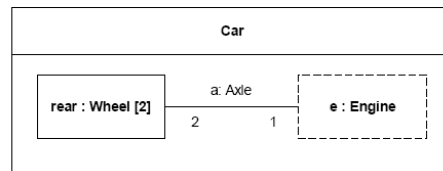


Diagrama de estructura compuesta



3.2.5 DIAGRAMA DE COMPONENTES

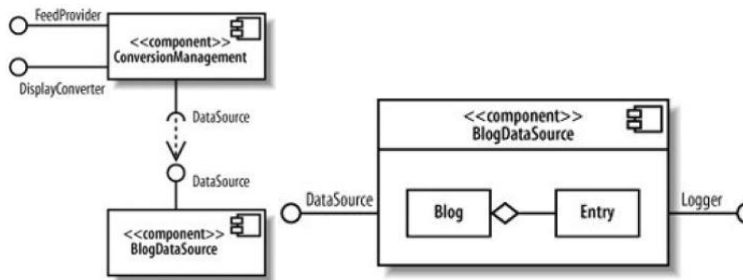
El diagrama de componentes proporciona una visión física de la construcción del sistema de información. Muestra la organización de los componentes software, sus interfaces y las dependencias entre ellos

Estos diagramas pueden incluir paquetes que permiten organizar la construcción del sistema de información en subsistemas y que recogen aspectos prácticos relacionados con la secuencia de compilación entre componentes, la agrupación de elementos en librerías, etc.

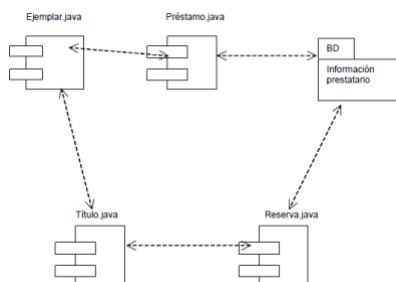
Notación "Lollipop" (piruleta): las dependencias de un clasificador a una interfaz pueden mostrarse mediante un semi-círculo abierto, que significa "interfaz requerida".

2 vistas de un componente:

- ✓ De caja negra: Muestra más de un componente. Mas adecuada para ver los componentes clave de un sistema y sus relaciones(fig izq)
- ✓ De caja blanca: Muestra la estructura interna de un solo componente. Realización interna del comportamiento externo. (fig centro)



3.2.6 DIAGRAMA DE DESPLIEGUE (O DIAGRAMA DE DISTRIBUCIÓN)



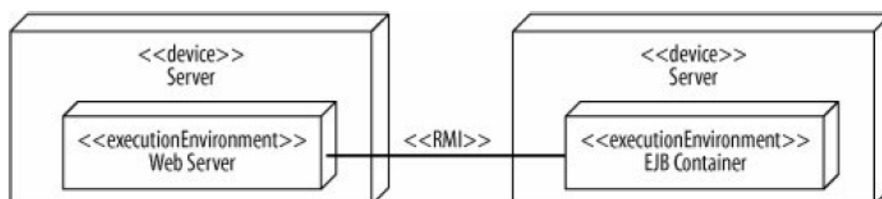
El objetivo de estos diagramas es mostrar la disposición de las particiones físicas del sistema de información y la asignación de los componentes software a estas particiones. Es decir, las relaciones físicas entre los componentes software y hardware en el sistema a entregar

Dos tipos de elementos:

- ✓ **Nodos** como partición física o funcional real, pero sin descender a detalles de infraestructura o

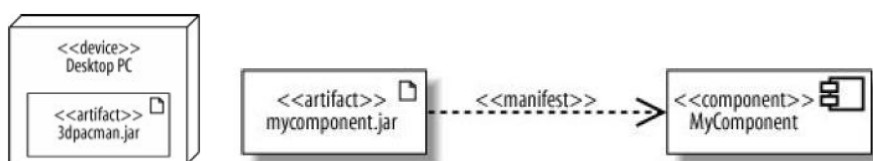
dimensionamiento

- Los **nodos HW** se estereotipan como `<<device>>` (dispositivo)
- Los **nodos SW** se estereotipan como `<<executionEnvironment>>` (entorno de ejecución)
- **Se le puede asignar un estereotipo a la comunicación indicando el protocolo**. El estereotipo debe ser del nivel más alto posible. Por ejemplo, si dudamos entre poner como estereotipo `<<TCP/IP>>` y `<<RMI>>`, como regla elegiremos el de más alto nivel, en este caso `<<RMI>>`.



- ✓ **Conexiones**, representan las formas de comunicación entre nodos
- ✓ cada nodo se le asocia un subsistema de construcción que agrupa componentes software, permitiendo de este modo, determinar la distribución de estos componentes. Por lo tanto, un diagrama de despliegue puede incluir, dependiendo del nivel de detalle, todos los elementos descritos en la técnica de diagrama de componentes, además los nodos y las conexiones propios de esta técnica.
- ✓ **Artefactos** Una pieza física de información que representan el software que se ubicará en los nodos.
 - Se representa como un rectángulo. Si no tiene el icono documento en la esquina superior derecha, será necesario estereotiparlo como `<<artifact>>`. (fig izq)
 - En versiones de UML anteriores a la 2.0 los componentes se podían ubicar en los nodos.
 - A partir de la versión 2.0 **en los nodos se ubican artefactos (no componentes)**.

- Para no hacer muy confuso el diagrama de despliegue, la relación entre el artefacto y el componente se mantiene fuera del diagrama
- Para relacionar artefactos y componentes se utiliza la relación de dependencia estereotipada con <<manifest>> (un artefacto manifiesta, revela un componente). Fig dcha.



3.2.7 DIAGRAMA DE INTERACCIÓN

El objetivo de esta técnica es describir el comportamiento dinámico del sistema de información mediante el paso de mensajes entre los objetos del mismo. Además representa un medio para verificar la coherencia del sistema mediante la validación con el modelo de clases

Los elementos que componen los diagramas de interacción son los objetos y los mensajes:

- **Un objeto es una entidad que tiene un estado**, un comportamiento e identidad.
- **Un mensaje es una comunicación entre dos objetos**. El envío de un mensaje por parte de un objeto (emisor) a otro (receptor), puede provocar que se ejecute una operación, se produzca un evento o se cree o destruya un objeto

2 tipos:

3.2.7.1 DIAGRAMA DE SECUENCIA

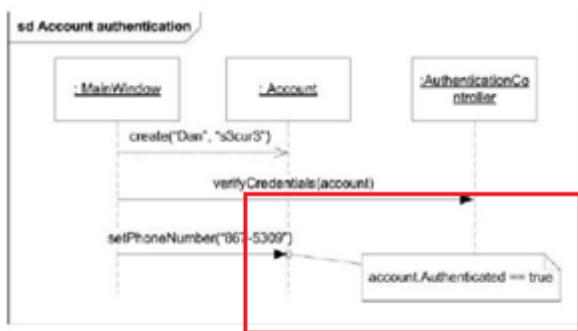
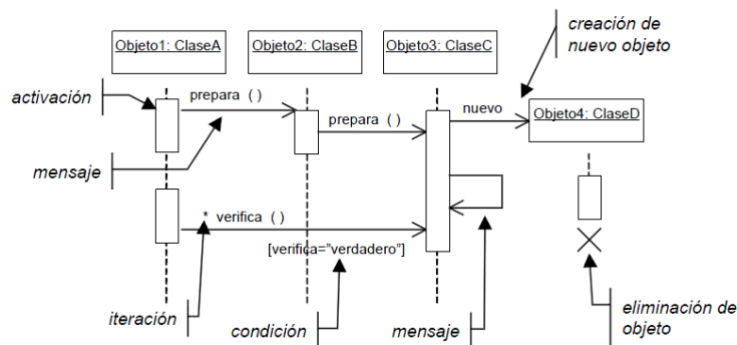
El diagrama de secuencia es un tipo de diagrama de interacción cuyo objetivo es describir el comportamiento dinámico del sistema de información haciendo énfasis en la secuencia de los mensajes intercambiados por los objetos

Un diagrama de secuencia tiene dos dimensiones, el eje vertical representa el tiempo y el eje horizontal los diferentes objetos. El tiempo avanza desde la parte superior del diagrama hacia la inferior.

Cada objeto tiene asociados una línea de vida y focos de control.

- ✓ La **línea de vida** indica el intervalo de tiempo durante el que existe ese objeto.
 - La línea de vida de un objeto puede desplegarse en dos o más líneas para mostrar los diferentes flujos de mensajes que puede intercambiar un objeto, dependiendo de alguna condición
- ✓ Un **foco de control o activación** muestra el periodo de tiempo en el cual el objeto se encuentra ejecutando alguna operación, ya sea directamente o mediante un procedimiento concurrente
 - Se representa como un rectángulo delgado superpuesto a la línea de vida del objeto.
 - Su largo dependerá de la duración de la acción.
- ✓ Un **mensaje** se representa como una flecha horizontal entre las líneas de vida de los objetos que intercambian el mensaje
 - Los mensajes pueden presentar también condiciones e iteraciones.
 - Una condición se representa mediante una expresión booleana encerrada entre corchetes junto a un mensaje

- **Síncrono o llamada:** El objeto que envía el mensaje queda bloqueado hasta que termina la llamada y recibe respuesta. 
- **Retorno.** Respuesta a un mensaje previo. 
- **Asíncrono.** El originario no espera respuesta y permanece activo pudiendo enviar mensajes 



Una **invariante** de estado es una restricción ubicada en una línea de vida que debe ser verdadera en el tiempo de ejecución (figura izq).

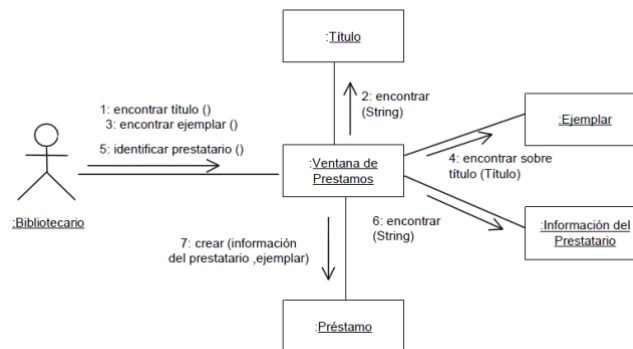
3.2.7.2 DIAGRAMA DE COLABORACIÓN

El diagrama de colaboración es un tipo de diagrama de interacción cuyo objetivo es describir el comportamiento dinámico del sistema de información mostrando cómo interactúan los objetos entre sí, es decir, con qué otros objetos tiene vínculos o intercambia mensajes un determinado objeto

En los diagramas de colaboración no existe una secuencia temporal en el eje vertical; es decir, la colocación de los mensajes en el diagrama no indica cuál es el orden en el que se suceden. Además, la colocación de los objetos es más flexible y permite mostrar de forma más clara cuáles son las colaboraciones entre ellos.

Elementos: objeto, vínculo y mensaje.

En estos diagramas la comunicación entre objetos se denomina **vínculo o enlace (link)** y estará particularizada mediante los mensajes que intercambian (el número de secuencia del mensaje va delante de su nombre).



3.2.8 DIAGRAMAS DE REVISIÓN DE INTERACCIONES (UML 2.4.X)

Es un diagrama que muestra cómo interactúan varios diagramas de interacciones. Este tipo de diagramas es muy útil para mostrar de qué manera distintos escenarios se combinan.

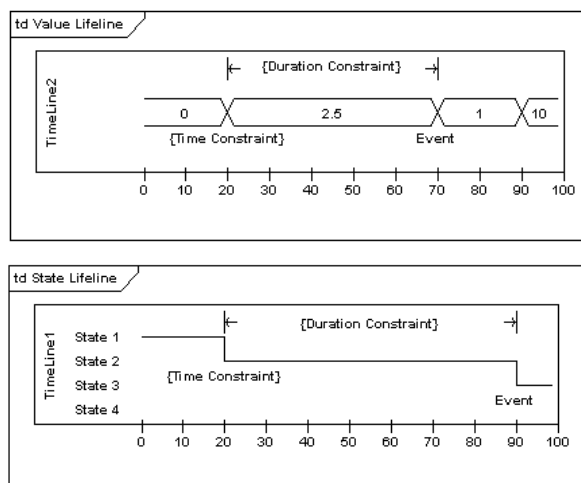
3.2.9 DIAGRAMA DE TIEMPOS (UML 2.X)

Se usan para mostrar el cambio en el estado o valor de uno o más elementos en el tiempo.

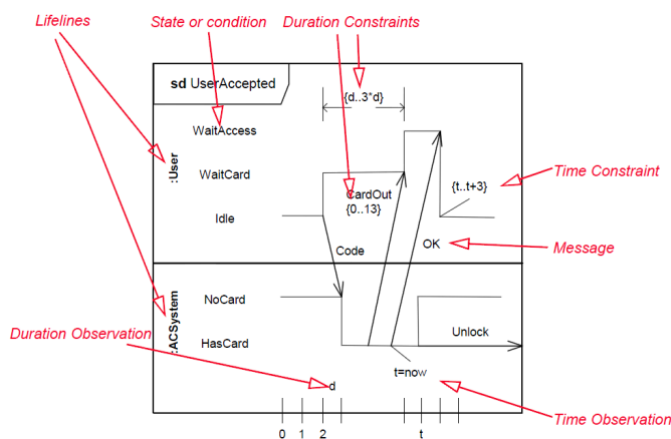
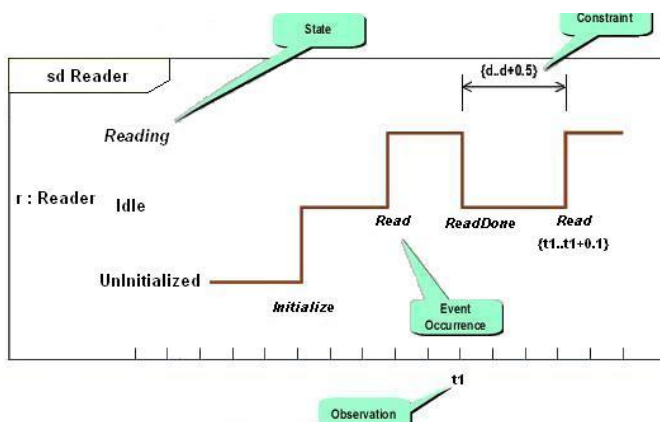
- ✓ Puede mostrar la interacción entre:
 - los eventos de tiempos / las restricciones de tiempos / la duración que los gobiernan.

Elementos:

- ✓ Tiene una o mas líneas de vida
 - **Línea de Vida de Estado** → Una línea de vida del estado muestra el cambio de estado de ítem en el tiempo (fig dcha)
 - **Línea de Vida del Valor** Representación **compacta** → Muestra el cambio del valor de un ítem en el tiempo (fig izq)



- ✓ El nombre del objeto se muestra en cada línea de vida
- ✓ Eje vertical -> Estados de la línea de vida
- ✓ Eje horizontal de tiempos
- ✓ Marcas de tiempo
- ✓ Eventos



3.2.10 DIAGRAMA DE GLOBAL DE INTERACCIÓN (UML 2.X)

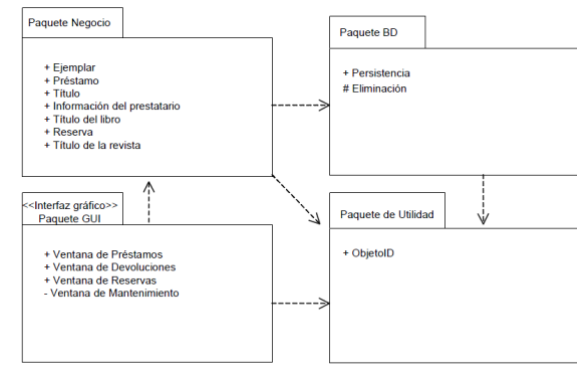
- ✓ Representa una vista global sobre el resto de diagramas de interacción (secuencia, comunicación y tiempo).
- ✓ Tiene la misma estructura que un diagrama de actividad, salvo que en vez actividades se representan interacciones.
- ✓ Es una especialización de los diagramas de actividad

3.2.11 DIAGRAMA DE PAQUETES

El objetivo de estos diagramas es obtener una visión más clara del sistema de información orientado a objetos, organizándolo en subsistemas, agrupando los elementos del análisis, diseño o construcción y detallando las relaciones de dependencia entre ellos.

El mecanismo de agrupación se denomina Paquete.

Los paquetes y sus dependencias son elementos de los diagramas de casos de uso, de clases y de componentes, por lo que se podría decir que el diagrama de paquetes es una extensión de éstos.

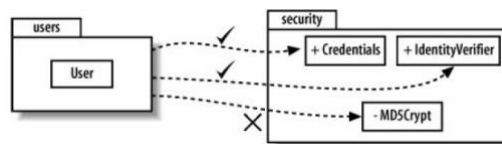


Importación: cuando un paquete importa otro, este puede usar los elementos del paquete importado sin tener que especificar el identificador de ámbito (namespace) del elemento. Cuando se importa un paquete, solo los elementos públicos están disponibles.

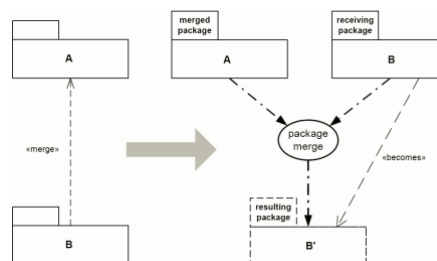
La importación, como otros elementos, tiene visibilidad:

- Pública (por defecto): los elementos importados tienen visibilidad pública en el paquete que los importa. Se estereotipa <<import>>.
- Privada: los elementos importados tienen visibilidad privada en el paquete que los importa. Se estereotipa <<access>>.

Exportación: modelado implícitamente a través de la visibilidad pública en los elementos de un paquete.



Una relación de fusión (merge) entre dos paquetes especifica que los contenidos de ambos paquetes son combinados.



3.2.12 DIAGRAMA DE TRANSICIÓN DE ESTADOS (UML 2.X DIAGRAMA DE MÁQUINA DE ESTADOS)

Un diagrama de transición de estados muestra el comportamiento dependiente del tiempo de un sistema de información.

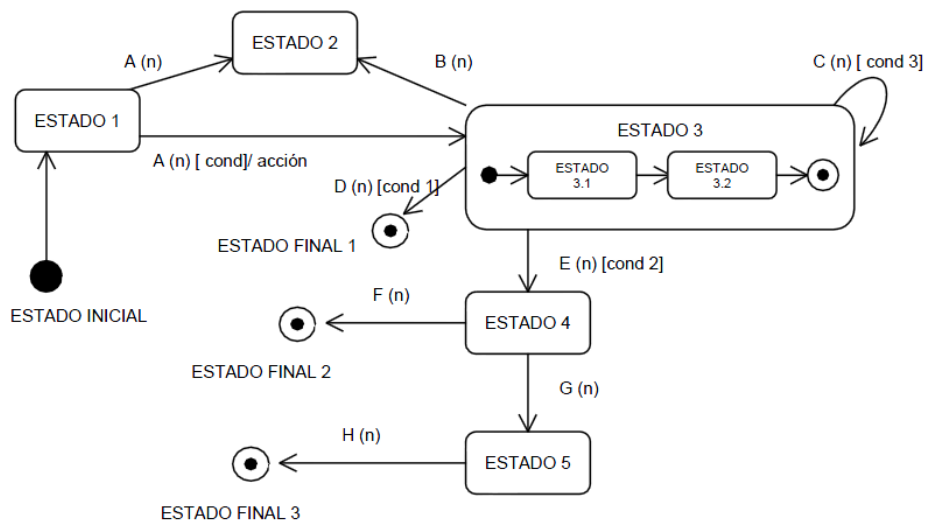
Representa los estados que puede tomar un componente o un sistema y muestra los eventos que implican el cambio de un estado a otro.

Elementos:

- ✓ **El estado de un componente** o sistema representa algún comportamiento que es observable externamente y que perdura durante un periodo de tiempo finito.
- ✓ Una **transición** es un cambio de estado producido por un evento y refleja los posibles caminos para llegar a un estado final desde un estado inicial

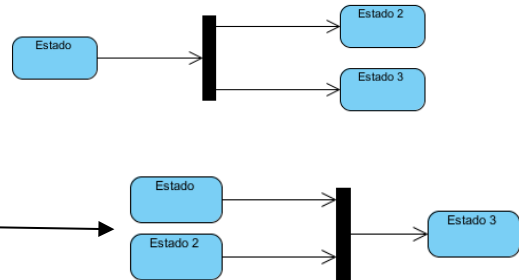
Reglas:

- ✓ Desde un estado pueden surgir varias transiciones en función del evento que desencadena el cambio de estado, teniendo en cuenta que, las transiciones que provienen del mismo estado no pueden tener el mismo evento, salvo que exista alguna condición que se aplique al evento.
- ✓ En ningún caso puede haber una transición dirigida al estado inicial.
- ✓ No se permiten transiciones que partan del estado final.
- ✓ Un sistema sólo puede tener un estado inicial, que se representa mediante una transición sin etiquetar al primer estado normal del diagrama. Pueden existir varias transiciones desde el estado inicial, pero deben tener asociadas condiciones, de manera que sólo una de ellas sea la responsable de iniciar el flujo.
- ✓ El estado final representa que un componente ha dejado de tener cualquier interacción o actividad.
- ✓ **Puede haber varios estados finales en un diagrama**, ya que es posible concluir el ciclo de vida de un componente desde distintos estados y mediante diferentes eventos, **pero dichos estados son mutuamente excluyentes**, es decir, sólo uno de ellos puede ocurrir durante una ejecución del sistema.
- ✓ Los diagramas de transición de estados comprenden además otros dos elementos que ayudan a clarificar el significado de los distintos estados por los que pasa un componente o sistema.
 - Una **acción es una operación instantánea asociada a un evento**, cuya duración se considera no significativa y que se puede ejecutar: dentro de un estado, al entrar en un estado o al salir del mismo.
 - Una **actividad es una operación asociada a un estado** que se ejecuta durante un intervalo de tiempo hasta que se produce el cambio a otro estado.
- ✓ Para aquellos estados que tengan un comportamiento complejo, se puede utilizar un diagrama de transición de estados de más bajo nivel. Estos diagramas se pueden mostrar por separado o bien incluirse en el diagrama de más alto nivel, dentro del contorno del estado que representa.



Los **Pseudo-estados** son una **abstracción del UML 2.x** para varios tipos de extremos usados en los Diagramas de Máquinas de Estados. Los Pseudo-estados se usan para expresar caminos de transición complejos:

- ✓ Inicial
- ✓ Terminar
- ✓ Puntos de Entrada
- ✓ Puntos de Salida
- ✓ Decisión
- ✓ Unión
- ✓ Bifurcación (fork node)
- ✓ Reunión (join node)
- ✓ Historia

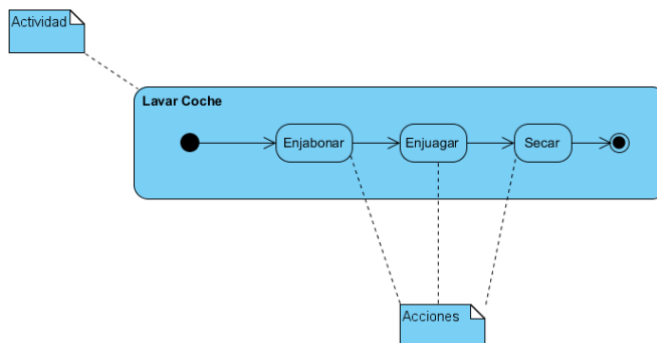


3.2.13 DIAGRAMA DE ACTIVIDAD

El Diagrama de Actividades es una variante de los Diagramas de Estados, **organizado con respecto a las acciones**.

Muestran un flujo de actividades → son “diagramas de flujo orientados a objetos”.

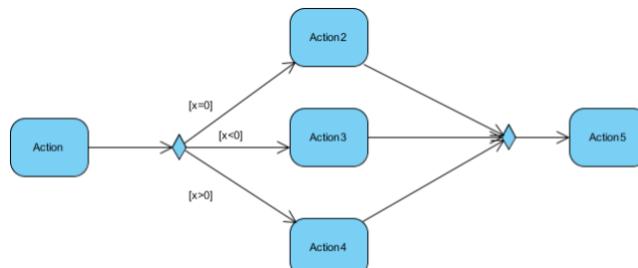
- ✓ **Nodo acción.** Ejecución de una acción atómica.
- ✓ **Nodo control:** gestionan el flujo de control dentro de una actividad
- ✓ **Nodo objeto:** son nodos especiales que indican que las instancias de un clasificador especial están disponibles en un punto específico en la actividad
- ✓ Una **actividad**. No es atómica, se compone de otras acciones y actividades



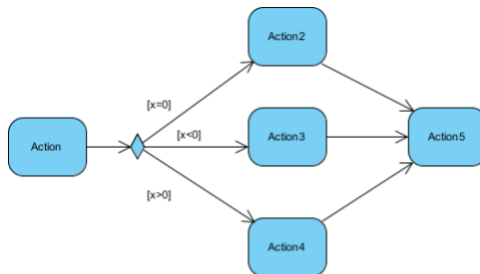
Las flechas dirigidas entre estados de acción representan **transiciones** con evento implícito.

- ✓ Podemos encontrar diferentes tipos de transacciones:
 - Secuencial o sin disparadores.
 - Decisiones (“decision node”) y fusiones (“merge node”) .
 - Junta varios flujos de ejecución (merge).
 - Se representa mediante un rombo como punto de llegada de varios flujos.
 - ¡Ojo!, en UML 1.x es común ver varios flujos que salen de un nodo decisión llegar directamente a la actividad, el merge está implícito. En UML 2.x hay que especificar el nodo unión porque **varios flujos llegando a una actividad**

significan que la misma no comenzará hasta que todos los flujos entrantes se hayan completado, cosa que no tiene sentido si vienen de un nodo decisión

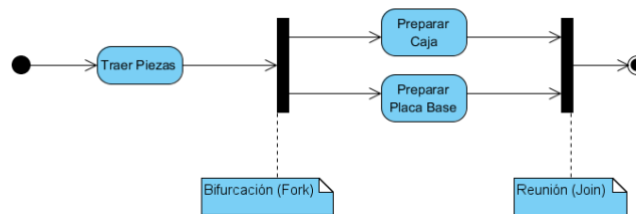


BIEN EN UML 2.X

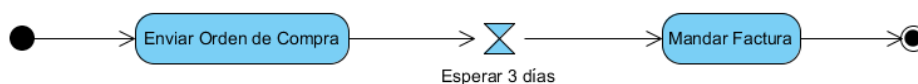


MAL EN UML 2.X

- Bifurcaciones ("fork node") y uniones ("join node")
 - La **bifurcación (fork)** permiten expresar la sincronización paralela de actividades. Las actividades invocadas después de una bifurcación (fork) son concurrentes.
 - La **unión / reunión (join)** expresa la confluencia concurrente de actividades

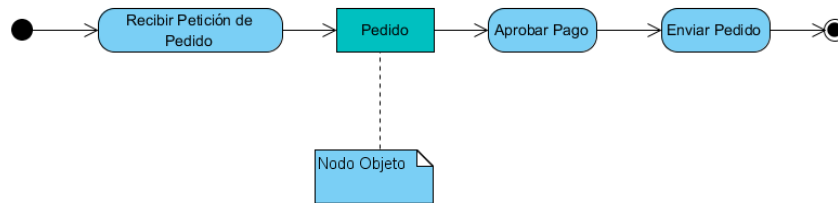


Eventos de tiempo Se utilizan para modelar periodos de tiempo entre actividades



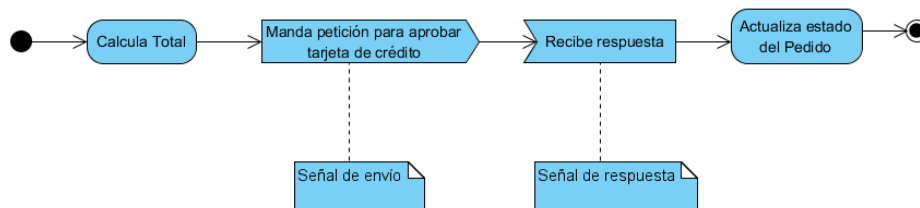
Los objetos pueden ser modelados dentro de los diagramas de actividad.

- ✓ Se usan nodos objetos para representar el flujo de datos a través de una actividad.
- ✓ Los objetos pueden ser creados, usados o modificados por las distintas acciones de la actividad.



Señales (se utiliza como llamada síncrona)

- ✓ Representan interacciones con participantes externos.
- ✓ Una señal recibida tiene el efecto de lanzar una acción dentro del diagrama de actividad



Nodo Inicial. La manera mas sencilla y común para comenzar una actividad es mediante un nodo

“inicial”.

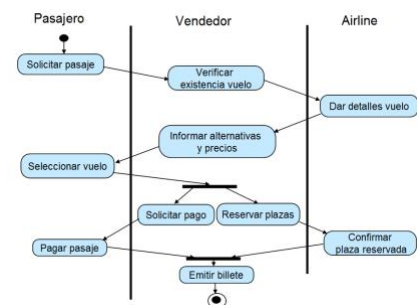
Nodo Finalizador de actividad. Cuando se alcanza un nodo finalizador de actividad, todas los flujos de esa actividad son finalizados.

Finalización de flujo. Se usa cuando no se quieren finalizar todas las acciones y flujos de una actividad, sino solo el flujo afectado.

Afecta solo al flujo que llega, y no al resto de flujos.

Calles (Swimlanes)

- ✓ Permiten ver QUIENES son los responsables de realizar las distintas actividades.
- ✓ Especificar qué parte de la organización es responsable de una actividad.
- ✓ Cada calle tiene un nombre único dentro del diagrama

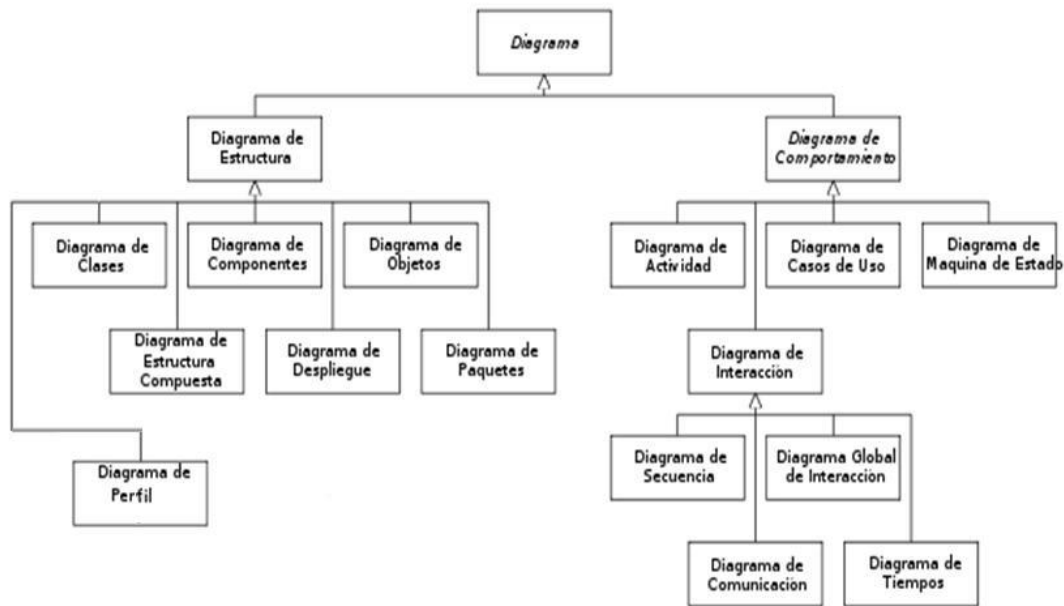


3.3 UML 2.X

XMI/Intercambio de diagramas: Permite compartir diagramas entre herramientas de modelado UML.

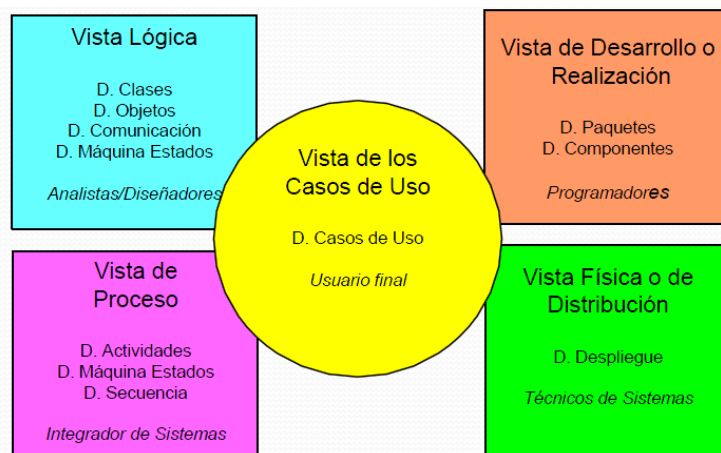
Tipos de diagramas UML:

- ✓ **Diagramas de estructura:** proporcionan una vista estática del sistema.
- ✓ **Diagramas de comportamiento:** proporcionan una vista dinámica del sistema (muestran un cambio progresivo).



Modelo de vistas “4+1” de Philippe Kruchten

- **Vista lógica:** Captura el vocabulario del ámbito del problema como un conjunto de clases y objetos. El énfasis está en mostrar cómo los objetos y las clases que componen un sistema implementan el comportamiento requerido del sistema.
- **Vista de proceso:** Modela los procesos en un sistema como clases activas. Es una variación orientada al proceso de la vista lógica y contiene los mismos artefactos.
- **Vista de implementación:** Modela los archivos y componentes sobre la base de código físico del sistema. También trata sobre ilustrar dependencias entre componentes y sobre la gestión de la configuración de conjuntos de componentes para definir una versión del sistema.
- **Vista de despliegue:** Modela el despliegue físico de artefactos en un conjunto de nodos físicos computacionales como ordenadores y periféricos. Le permite modelar la distribución de artefactos en los nodos de un sistema distribuido.
- **Vista de caso de uso:** Captura los requisitos básicos para el sistema como un conjunto de casos de uso. Estos casos de uso proporcionan la base para la construcción de otras vistas.



Qué modela cada diagrama:

- ✓ **Casos de uso:** interacciones entre el sistema y usuarios o sistemas externos. Requerimientos funcionales.
- ✓ **Actividad:** actividades secuenciales o paralelas del sistema. Flujos de trabajo.
- ✓ **Secuencia:** interacciones entre objetos en las que el orden de las interacciones es importante.
- ✓ **Comunicación:** es semánticamente equivalente al diagrama de secuencia. Llamado diagrama de colaboración en UML (1.x). Mientras que el diagrama de secuencia se organiza de acuerdo al tiempo, el diagrama de colaboración se organiza de acuerdo al espacio.
- ✓ **Tiempos:** muestra el cambio en el estado o valor de uno o más elementos en el tiempo.
- ✓ **Global de interacción:** agrupación de los diagramas de secuencia, comunicación y tiempo.
- ✓ **Clases:** clases, interfaces y relaciones entre ellos.
- ✓ **Objetos:** instancias de las clases del diagrama de clases que son importantes para el sistema.
- ✓ **Estructura compuesta:** muestra la estructura interna de una clase.
- ✓ **Componentes:** componentes importantes del sistema y los interfaces para interactuar con ellos.
- ✓ **Paquetes:** organización jerárquica de clases y componentes.
- ✓ **Máquina de estados:** estados de un objeto a lo largo de su ciclo de vida y los eventos que pueden cambiar ese estado.
- ✓ **Despliegue:** cómo se despliega el sistema en el mundo real. Ubicación del SW en el HW. También se le conoce como diagrama de distribución.
- ✓ **Perfil:** se utiliza para proveer un mecanismo de extensión al UML estándar. Está basado en la definición de estereotipos, restricciones y valores etiquetados personalizados.

Un metamodelo:

- ✓ Es un modelo que define el lenguaje para expresar un modelo.
- ✓ Describe todos los conceptos que pueden usarse en el modelo.
- ✓ Cada elemento de un modelo es una instancia de una metaclase en el metamodelo.
- ✓ Los lenguajes de modelamiento como UML tienen, en general, una sintaxis gráfica que requiere otro tipo de descripción, descripción que se da mediante un metamodelo.
- ✓ **MOF (Meta Object Facility)** es un **meta-meta-modelo** que permite definir meta-modelos (como UML).
- ✓ **UML** es un **meta-modelo** que permite definir modelos. Es una instancia de MOF

3.4 MECANISMOS DE EXTENSIBILIDAD

Los Perfiles UML son el mecanismo que proporciona el propio UML para extender su sintaxis y su semántica para expresar los conceptos específicos de un determinado dominio de aplicación. Cuando modelamos un perfil estamos trabajando con el meta-modelo de UML

Un Perfil se define en un paquete UML, estereotipado <<profile>>, que extiende a un metamodelo o a otro Perfil

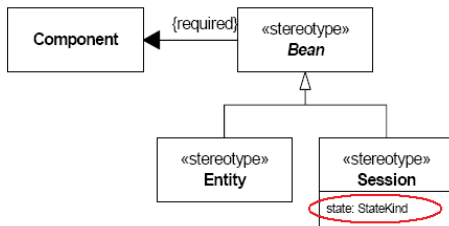
Mecanismos para definir perfiles:

- ✓ **Un estereotipo** permite indicar especificaciones del lenguaje al que se refiere el diagrama de UML (p.e., un paquete de estereotipo <<subsistema>> o una clase <<persistente>>).
- ✓ **Una restricción (constraints)** identifica un comportamiento forzado de una clase o relación. Mediante la restricción se está forzando el comportamiento que debe tener el objeto al que se le aplica. Se pueden representar
 - En lenguaje natural
 - En la gramática formal OCL. (Object Constraint Language – Lenguaje de Restricciones de Objetos).
 - OCL es un lenguaje para consultar y restringir los elementos de un modelo, definido por OMG y es parte integrante de UML
 - Mediante una nota



- ✓ **Un valor etiquetado** extiende las propiedades de un bloque de construcción UML, añade nueva información en la especificación de ese elemento (p.e. versión de un componente). → Los valores etiquetados son las propiedades de los estereotipos.

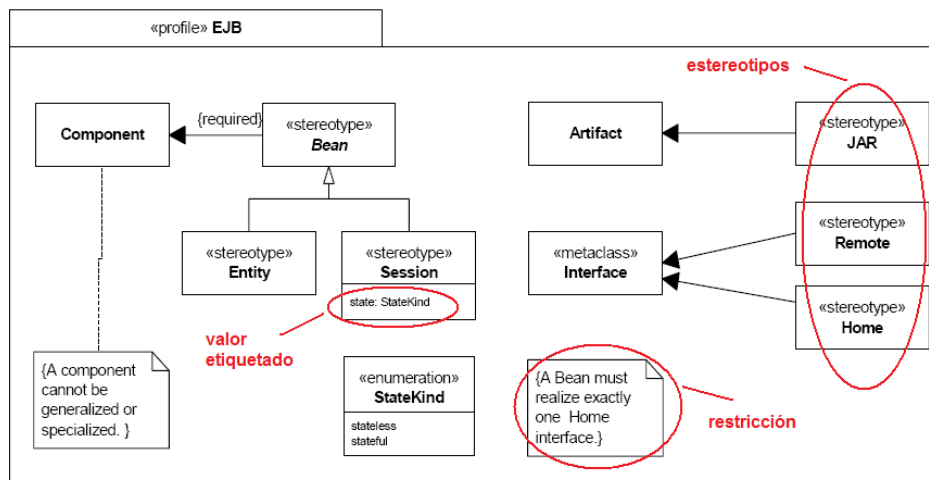
- es un meta-atributo adicional que se asocia a una meta-clase del metamodelo extendido por un Perfil.



El estereotipo **Bean** es abstracto (cursiva), lo que implica que no se podrá instanciar. Se instanciarán los estereotipos concretos como **Entity** y **Session**. La extensión {required} se aplica a todos los estereotipos concretos que heredan de **Bean**

3.5 ELEMENTOS COMUNES A CUALQUIER DIAGRAMA UML

- Las notas: sirven para añadir cualquier tipo de comentario a un diagrama o a un elemento de un diagrama.
- Las dependencias: entre dos elementos de un diagrama significa que un cambio en el elemento destino puede implicar un cambio en el elemento origen



Tipos primitivos de datos en UML → ojo no hay CHAR

Integer	An instance of Integer is a value in the (infinite) set of integers (...-2, -1, 0, 1, 2...).
Boolean	An instance of Boolean is one of the predefined values <i>true</i> and <i>false</i> .
String	An instance of String defines a sequence of characters. Character sets may include non-Roman alphabets. The semantics of the string itself depends on its purpose; it can be a comment, computational language expression, OCL expression, etc.
UnlimitedNatural	An instance of UnlimitedNatural is a value in the (infinite) set of natural numbers (0, 1, 2...) plus <i>unbounded</i> . The value of <i>unbounded</i> is shown using an asterisk (*). UnlimitedNatural values are typically used to denote the upper bound of a range, such as a multiplicity; <i>unbounded</i> is used whenever the range is specified to have no upper bound.
Real	An instance of Real is a value in the (infinite) set of real numbers. Typically an implementation will represent Real numbers using a floating point standard such as IEEE 754.

4 BPM

Se llama de Business Process Management (BPM) a la metodología empresarial cuyo objetivo es mejorar la eficiencia a través de la gestión sistemática de los procesos de negocio, que se deben modelar, automatizar, integrar, monitorizar y optimizar de forma continua.

- ✓ Se enfoca en la administración de los procesos de negocio
- ✓ Ayuda a optimizar o adaptar los procesos de la empresa al mercado o a la organización.
- ✓ Su objetivo es mejorar la eficiencia a través de la gestión sistemática de los procesos de negocio soportando su ciclo completo; modelar, automatizar, integrar, ejecutar, monitorizar, optimizar.

BPM trata sobre el ciclo de vida de los procesos de negocio mientras que **SOA** se centra en ver los procesos como servicios que deben ser expuestos en forma débilmente acoplada y consumidos por app compuestas.

Un **proceso de negocio** no es más que un **conjunto de tareas o actividades** relacionadas lógicamente que **permiten crear valor** transformando una entrada en una salida logrando así un resultado de negocio definido

BPM Suite une lo mejor del mundo del WorkFlow Tradicional con el ESB.

En un mismo flujo se integran tareas realizadas por personas, con tareas automatizadas (sistemas), entregándole al participante de un proceso de negocio una sola interfaz, ocultando la interacción con los sistemas legados.

Las otras características de valor agregado de BPM son: que ofrece un grupo de herramientas para atender todo el ciclo de vida de un proceso de negocio, dentro de las cuales se destaca, el modelador (diseño procesos), el IDE (desarrollo de servicios), y el monitor (BAM)

BAM Business Activity Monitoring: proporciona monitorización de ejecución de procesos, visión en tiempo real y control de negocio.

Nombre	Organismo	Campo de aplicación	Notación	Formato intercambio	
XPDL	WfMC	Lenguaje de ejecución	No	Sí	http://www.wfmc.org/standards/xpdl.htm
BPEL	OASIS	Lenguaje de ejecución	No	Sí	http://www.oasis-open.org/committees/wsbpel/
BPMN	BPMI	Análisis de Procesos de Negocio	Sí	No	http://www.bpmn.org/
BPML	BPMI	Lenguaje de ejecución	No	Sí	[Discontinuado]

4.1 BPMN

Define una notación para la definición de procesos de negocio, lo que es una plataforma independiente con respecto a definiciones específicas

Modela tanto la secuencia de actividades como los datos o mensajes intercambiados entre los distintos participantes. BPMN no está pensado para modelar aplicaciones, sino procesos que correrán dentro de dichas aplicaciones

Con **BPML** disponemos de la traducción de un formato gráfico (para ser leído por personas) a un formato legible por máquinas (y permitir el intercambio entre distintas herramientas).

4.2 XPDL (XML PROCESS DEFINITION LANGUAGE)

Las especificaciones XPDL y BPMN afrontan el mismo problema desde diferentes perspectivas. XPDL proporciona un formato de fichero XML para ser intercambiado entre aplicaciones. BPMN proporciona una notación gráfica para facilitar la comunicación humana entre usuarios". XPDL y BPMN forman un binomio sobre todo gracias a la existencia de herramientas que permitan dibujar en BPMN e intercambiar esos diagramas en XPDL.

XPDL especifica un formato de diseño de los procesos. Permite una representación gráfica de los procesos incluyendo coordenadas X e Y para cada nodo implementado.

XPDL en su representación de BPMN si es transportable de una herramienta a otra (ya que luego cada proveedor realiza su conversión de XPDL a BPEL según necesite).

4.3 CADENA DE VALOR BPMN-XPDL-BPEL

1. El usuario genera un diagrama con notación BPMN del proceso que quiere construir.
2. Se guardan los diagramas parciales como XPDL durante el desarrollo. En el caso de XPDL 2.0 esto es inmediato.
3. Finalmente se traducen ciertas partes a BPEL (básicamente en lo que a intercambio de datos se refiere, no a interacciones humanas donde BPEL no es adecuado ya que necesita de módulos específicos que son diferentes en su implementación por los distintos fabricantes, para eso está **BPEL4People**).

4.4 LENGUAJES DE ORQUESTACIÓN

Un nivel más abajo de este binomio nos encontramos con BPEL (Business Process Execution Language), un **lenguaje de orquestación** de servicios web que se está convirtiendo en un estándar de facto.

BPMN adquiere aún más importancia al poder realizarse transformaciones directas desde el nivel gráfico, el propio BPMN, hasta el nivel de implementación en servicios web con BPEL. Además existe la posibilidad de un paso intermedio expresando el proceso de negocio en XPD.

El lenguaje de programación BPEL también posibilita:

- ✓ Un mecanismo de correlación de mensajes basado en propiedades.
- ✓ Variables del tipo XML y WSDL.
- ✓ Un modelo de lenguaje extensible de componentes para permitir escribir expresiones y consultas (queries) en múltiples lenguajes: BPEL soporta Xpath 1.0 predeterminadamente.
- ✓ Construcciones de programación estructurada incluyendo "if-then-elseif-else", "while", "sequence" (posibilita la ejecución de comandos en orden) y "flow" (posibilita la ejecución de comandos en paralelo).
- ✓ Un sistema de ámbito (scoping) que permite el encapsulamiento de lógica con variables locales y manejadores de eventos.
- ✓ Ámbitos serializados para controlar los accesos a las variables.

BPEL4People es una extensión realizada para soportar una gran cantidad de escenarios que involucran gente con procesos de negocio.

BPEL no tiene soporte gráfico; es decir, no especifica como deben ser los diagramas interpretativos de los procesos que define. El objetivo de BPEL es ofrecer una forma de orquestar servicios web, la secuencia de interacciones subyacente y el flujo de datos punto a punto. → se utiliza BPMN
→ **BPMN2BPEL**

YAWL (Yet Another Workflow Language)

Los desarrolladores de YAWL decidieron tomar **Redes de Petri** como un punto de partida y extender esta formalización con tres constructores principales, nombrado or-join, grupos de cancelación, y actividades multi-instancia

Lenguajes de coreografía WSCDL y WSCI

BPR → orientado a realizar cambios drásticos en los procesos de negocio de una organización para mejorar su eficiencia